面向对象程序设计课程大作业　方向 2.3：超长记忆增强

Mandol 超长记忆增强

从源码理解到记忆生命周期管理设计

目标系统仓库：[https://github.com/AgentCombo/Mandol]

研究方向	汇报学生与学号	指导教师
方向 2.3：超长记忆增强	王浚沣　2024K8009909018	王伟

系统现在怎么工作？

↓

第一部分：读懂现有 Mandol

MemoryUnit · MemorySpace · SemanticMap · SemanticGraph · 三种记忆塔 · 写入与检索

↓

第二部分：扩展方案

问题定位 · 价值评分 · 重复检测 · 冲突版本 · 生命周期状态 · 实验验证

↓

如何让它适合长期运行？

Mandol 把对话变成可组织、可检索的长期记忆

为什么需要记忆

大模型上下文窗口有限；

历史对话不会自动永久保留；

用户信息分散在多轮对话中。

Mandol 做什么

保存原始对话与结构化记忆；

生成稠密与稀疏向量索引；

维护语义图关系并支持多路检索。

最终目的

在合适的时机找回相关记忆；

控制上下文噪声与 Token 开销；

帮助智能体持续理解用户。

1. 用户当前问题 → 2. Mandol 多路记忆检索 → 3. 组装大模型上下文

总结：Mandol 是一个面向智能体的记忆组织、存储与检索系统。

系统主线

第一条线：消息写入 (Write)

入口：`MandolService`

接收多轮聊天消息

封装为标准 `MemoryUnit`

写入基础切片层（`raw / l0`）

第二条线：记忆加工 (Build)

核心：三种 `AutoBuilder`

从 `L0` 提炼 `L1/L2` 摘要、

情景事件（`episodic`）与

实体关系（`entity`）

第三条线：找回记忆 (Retrieve)

核心：`TripleTowerRetriever`

融合三塔多路检索结果

经 `CascadeConfidencePruner`

按预算裁剪输出上下文

⇓ ⇓ ⇓

统一内存语义底座：MemoryUnit + MemorySpace + SemanticMap + SemanticGraph

所有原始对话、摘要、事件与实体均统一表示为 MemoryUnit，共享同一套向量索引与语义图存储

MemoryUnit 是所有记忆的统一对象

属性字段	属性含义	在系统中的核心作用
`uid`	唯一标识符	精确区分与索引每一条独立记忆
`raw_data`	原始载荷内容	保存文本、图像或其他多模态交互数据
`metadata`	来源、层级、会话等元信息	说明记忆从哪里来、属于哪一类空间
`embedding`	稠密语义向量	支持语义相似度检索与近邻重复检测
`sparse_embedding`	稀疏词项向量	支持关键词精确匹配与稀疏检索
`created_time`	创建时间戳	支持时序关系分析与时间新鲜度衰减计算

统一表示能力：可表示原始对话，也可表示摘要、事实、事件或实体。 **图节点基石**：它是 `SemanticGraph` 中所有节点内容的基础数据来源。

总结：MemoryUnit 封装了“单条记忆”的数据和基本行为。

MemorySpace 负责组织记忆，不负责保存正文

空间名称 (Space)	管理的记忆类型	功能定位与理解方式
raw	原始对话记录	最接近用户原始输入，保留完整上下文痕迹
l0	基础对话切片	三类自动构建器（AutoBuilder）的直接输入源
l1 / l2	会话摘要与高层总结	用更精炼的抽象节点表达更长跨度的信息
episodic	具体事实与事件	聚焦时间线与行为，回答“发生了什么”
entity	核心实体与属性信息	聚焦人物、概念与关联，回答“谁和谁有什么关系”

成员管理

添加或移除记忆 UID 集合

归属判定

判断某条记忆是否属于当前空间

层级遍历

保存子空间关系，支持递归遍历

简单理解：MemorySpace 像文件夹（只管目录结构）；MemoryUnit 是文件；SemanticMap 统一管理文件内容。

SemanticMap 管内容、空间、索引和访问统计

核心成员结构	主要职责	与本课题（超长记忆增强）的关系
`memory_units`	保存 UID 到 MemoryUnit 的映射	决定哪些记忆实体当前仍然驻留内存
`memory_spaces`	管理各命名空间及其成员关系	决定记忆属于哪一逻辑层级或状态空间
`dense / sparse index`	维护稠密向量与稀疏向量索引	支持高效近邻检索与跨会话语义重复检测
`access counts`	记录每条记忆被检索命中的次数	直接反映记忆的使用频率（常用度指标 A）
`last accessed`	记录每条记忆最近一次被访问的时间	用于计算记忆的时间新鲜度与遗忘衰减（指标 R）

现有基础：系统已经记录了部分“价值评估”所需的数据，但还没有把这些数据组合成长期维护策略。

局限性：现有 deduplicate_units 主要处理 UID 层面的重复，不等于跨会话语义去重。

SemanticGraph 在记忆节点之上表达关系

图中的节点 (Nodes)	图中的边 (Edges)	底层承载对象
原始对话、L0、L1、L2 节点	来源溯源关系、时间先后关系	`MemoryUnit`
事实节点、事件节点、实体节点	实体关联边、事件参与关系边	`MemoryUnit`
未来扩展：新旧事实版本节点	`supersedes` 等版本演进边	`MemoryUnit + metadata`

SemanticGraph 记忆关系拓扑示意



总结：SemanticMap 负责“记忆是什么”；SemanticGraph 负责“记忆之间有什么联系”。

一条聊天消息怎样写入系统

1. MandolService.add_chat_message()

⇓

2. 创建 uid、raw_data 与 metadata

⇓

3. 实例化 MemoryUnit，绑定当前 session

⇓

4. SemanticGraph.add_unit()

⇓

5. SemanticMap.add_unit() 生成向量并更新索引

MandolService（业务入口）

对外暴露对话接入 API，负责参数封装与会话绑定。

SemanticGraph（写入协调者）

协调节点入图与拓扑关系维护，转发底层存储请求。

SemanticMap（核心存储与索引）

实际保存 `MemoryUnit`、分配 `MemorySpace` 并更新向量索引。

三种自动构建器从 L0 生成不同类型的记忆

自动构建器 (AutoBuilder)	生成结果类型	负责回答的核心问题
`HierarchicalAutoBuilder`	L1 会话摘要、L2 高层总结	这段对话主要讲了什么？
`EpisodicAutoBuilder`	事实节点、事件、时间信息	发生了什么？什么时候发生？
`EntityRelationAutoBuilder`	实体节点、实体间关系边	谁和谁有什么关系？

L0 基础对话切片 ⟹ **三种 AutoBuilder 并行提炼结构化信息** ⟹ **统一封装为 MemoryUnit 写入语义图**

共同点：三个构建器最终都把结果转换成 MemoryUnit，再写入同一套语义后端。

三种记忆塔逻辑分工，物理共享

层次记忆塔 (Hierarchical)

核心对象：摘要与高层总结

负责宏观脉络归纳

回答“讲了什么”

情景记忆塔 (Episodic)

核心对象：事实与具体事件

负责时序状态与经历追踪

回答“发生了什么”

实体关系塔 (Entity-Relation)

核心对象：实体与关联边

负责结构化知识图谱连接

回答“谁与谁有关”

⇓ ⇓ ⇓

物理共享同一底层：三类结果 ⟶ MemoryUnit ⟶ SemanticMap + SemanticGraph

逻辑上按三种视角分工组织，物理上统一存储于同一内存图与混合索引中，避免跨库 I/O 割裂

注意：三种记忆塔不是三个完全独立的数据库，而是共享同一个图和索引后端。

检索阶段融合多路结果，再按预算裁剪

1. TripleTowerRetriever
路由层次、情景、实体三塔检索

⟹

2. MultiRetriever
融合 BM25、稠密/稀疏向量与图检索

⟹

3. CascadeConfidencePruner
去重、消冲突、控多样性与预算

检索核心组件	在检索链路中的核心职责
`TripleTowerRetriever`	根据查询需求，协同路由层次、实体关系、情景三类记忆塔的检索通道
`MultiRetriever`	在单塔内部融合 BM25、稠密向量、稀疏向量和语义图多跳检索结果
`CascadeConfidencePruner`	在组装上下文前处理候选集重复、冲突、多样性与 Token 窗口上限

现有能力局限性：检索时裁剪是“这次查询不要带什么”；生命周期维护是“以后系统还要不要长期保留什么”。

长期运行后，系统会遇到四类记忆管理问题

问题编号	长期运行痛点现象	现有机制的能力边界	本课题需要补齐的能力
P1	不同会话重复表达同一事实	UID 不同，基础去重识别不到	**跨会话语义重复检测与合并**
P2	事实变化后新旧状态同时命中	仅有单次检索裁剪，无版本管理	**当前有效版本与历史版本链维护**
P3	低价值内容长期占据核心索引	冷热存储仅改变物理存放位置	**多维信息价值评估与软归档降级**
P4	摘要、原文和事实越来越冗余	构建流程只增不减，无维护状态	**全生命周期状态机与来源追溯维护**

新增 MemoryLifecycleManager，作为维护层接入现有系统

现有写入主线：MandolService ⟶ SemanticGraph ⟶ SemanticMap

⇓ **触发时机：会话结束或达到批量写入阈值**

新增维护协调层：MemoryLifecycleManager

统一调度价值评分、跨会话重复检测、事实冲突消解与生命周期策略执行

⇓

ValueScorer	RedundancyDetector	ConflictResolver	MaintenancePolicy
计算单条记忆综合价值	查找跨会话高相似冗余记忆	处理事实变更与新旧版本边	决定合并、压缩、保护或归档

改进原则：不替换 Mandol 的底层存储和检索；优先复用 SemanticMap、SemanticGraph 的已有接口。

记忆价值由内容、使用情况和图结构共同决定

价值评估指标	核心衡量含义	在 Mandol 中的实现数据依据
语义重要性 I	是否包含用户偏好、长期约束或关键事实	记忆空间类型（`metadata`）、关键词与规则特征
访问频率 A	该记忆在历史查询中是否经常被检索命中	直接复用 `SemanticMap` 的 `access counts` 统计
时间新鲜度 R	最近是否被访问、更新或刚刚创建	结合 `last accessed` 与 `created_time` 时间差
图中心性 C	在语义图中是否连接诸多核心实体或事实	统计 `SemanticGraph` 中节点的关联边数量（度数）
重复度惩罚 D	系统中是否已存在表达相同语义的更优记忆	基于 `embedding` 近邻余弦相似度最大值

综合价值评估公式： $V(m) = \alpha I + \beta A + \gamma R + \delta C - \lambda D$

实验原则：先定义指标，再通过消融实验比较不同权重；不在没有数据时虚构效果。

重复内容可以合并，事实冲突必须保留版本

重复记忆 (Redundancy)	事实冲突 (Fact Conflict)	统一维护处理原则
向量相似度高、主题一致、无明显时间状态冲突	例如：“用户住在北京”后来变成“用户搬到了上海”	**合并同义重复，但绝不盲目覆盖冲突历史**
保留代表性主节点，合并来源 UID 并更新索引	在图中建立“上海 $\xrightarrow{\text{supersedes}}$ 北京”的版本替代边	在 metadata 中记录 valid_from、valid_to、is_current



方案价值：系统既能回答当前状态，也能解释历史变化，避免把“变化”误判成“重复”。

用生命周期状态管理记忆的长期变化

PROTECTED 重点保护 ⟵ ACTIVE 正常活跃 ⟶ CANDIDATE 待维护候选 ⟶ COMPRESSED 合并压缩 / ARCHIVED 软归档

生命周期状态标识	状态定义与系统检索行为
ACTIVE	正常活跃状态，全量参与常规多路检索与图遍历
CANDIDATE	疑似重复或综合价值下降，进入维护候选池等待进一步合并或归档判断
PROTECTED	核心偏好、关键事实或高中心性枢纽节点，受保护不轻易裁剪清理
COMPRESSED	已将其核心信息合并至代表性节点或高层摘要节点，原节点退出主索引
ARCHIVED	暂时退出主检索通道以节省开销，但保留原始数据与溯源链，支持按需恢复

为什么要有 ARCHIVED：归档不等于物理删除；它保留了未来恢复和解释系统决策的可能。

新增类各司其职，通过已有接口完成维护

维护控制器：MemoryLifecycleManager

统一编排生命周期维护流水线，对外提供简洁的批量维护入口

⇓

ValueScorer | RedundancyDetector | ConflictResolver | MaintenancePolicy

⇓

SemanticMap：读取访问统计、查询向量近邻、更新索引

SemanticGraph：维护节点拓扑关系与 `supersedes` 版本边

面向对象设计原则	在本扩展方案中的具体体现
封装 (Encapsulation)	价值评分、重复检测、冲突版本处理与状态转换判断均由独立类封装内部细节
单一职责 (SRP)	每个扩展类仅专注解决一个子问题，逻辑边界清晰，便于独立编写单元测试与替换
开闭原则 (OCP)	调整评分公式或替换合并策略时，无需修改 `TripleTowerRetriever` 等原有检索代码

实验同时观察“压缩收益”和“检索是否受损”

消融实验组别	开启的维护模块内容	核心观察与评测指标
1. 原始 Mandol (Baseline)	不启用生命周期维护	记忆节点总量、Recall@K、P95 检索延迟
2. 仅价值评分 (+Scorer)	只计算记忆价值分布，不执行合并	低价值记忆识别比例、单次评估耗时
3. 评分 + 合并 (+Merge)	合并跨会话高相似冗余记忆	重复率下降幅度、节点压缩率、上下文 Token 节省量
4. 完整方案 (Full Ours)	开启评分、合并、冲突版本与软归档	最新事实命中率、历史事实可追溯率、综合延迟

1. 配套单元测试

覆盖重复输入、空输入、极端时间戳与冲突事实边界用例

2. 图结构一致性校验

验证合并与归档后零悬空边，来源 UID 链条完整可追溯

3. 端到端长程评测

验证长期运行后存储规模显著下降，且检索质量不退化

判断原则：不是只看“删了多少”，而是同时看存储规模、检索质量、上下文长度和维护成本。

在 Mandol 的现有底座上增加可验证的记忆生命周期管理

第一部分：我已经理解的现有系统	第二部分：我准备实现的扩展方案
`MemoryUnit`：所有记忆的统一封装对象	`MemoryLifecycleManager`：低侵入生命周期维护入口
`SemanticMap`：管理内容、空间、索引与访问统计	`ValueScorer`：融合五维特征计算记忆保留价值
`SemanticGraph`：表达节点关系与支持图检索	`RedundancyDetector / ConflictResolver`：处理去重与版本边
三种记忆塔：从层次、情景、实体多角度加工记忆	`MaintenancePolicy`：执行保护、合并、压缩与软归档

下一步工作计划

先完成 ValueScorer 评分与 RedundancyDetector 重复检测单元测试，再接入会话结束后的批量维护与对比实验

谢谢聆听 · 敬请老师和同学提出宝贵意见